

TPML: Task Planning for Multi-UAV System with Large Language Models

Jinqiang Cui¹, Guocai Liu¹, Hui Wang¹, Yue Yu^{1,2}, Jiankun Yang¹

Abstract—Efficient task planning is pivotal for multi-UAV systems navigating dynamic environments. Traditional task planning methods face challenges in adapting to the constantly changing scenarios. The emergence of large language models (LLMs) offers promising solutions to bridge this gap. Our proposal, TPML, leverages LLMs as a command interface to comprehend operators' intentions and translate them into executable codes. Harnessing the creative capabilities of generative models, TPML can command multiple UAVs in both synchronous and asynchronous patterns with a single natural-language input. Experimental results are initially validated in a tailored simulation environment before transitioning to practical implementations. Successful demonstrations of both synchronous and asynchronous missions in real-world scenarios underscore the efficacy of TPML.

I. INTRODUCTION

In the dynamic realm of unmanned aerial vehicles (UAVs), the convergence of cutting-edge technologies has brought a new era of possibilities. This paper presents an innovative approach to the task planning of multi-UAV systems by harnessing the power of large language models (LLMs) in order to achieve interactive and efficient task planning and coordination. As UAVs become integral to various applications, ranging from surveillance to delivery services, the demand for sophisticated and adaptable swarm control mechanisms has surged. The current landscape presents challenges in achieving seamless task planning and coordination, particularly in scenarios that demand both synchronous and asynchronous execution.

Despite various strides in UAV technology, existing task planning algorithms for multi-UAV systems grapple with the complexities of coordinating tasks efficiently within a swarm, especially in the a constantly-changing environment where instant interaction with the operator are required. First, the coordination and synchronization of multiple UAVs to achieve a common goal is complicated. Each UAV has its own capabilities, constraints, and sensing modalities, which need to be taken into account when designing control strategies. The second challenge lies in task allocation and path planning for multi-UAV systems. Efficiently assigning tasks to UAVs and planning their trajectories while considering various constraints, such as energy consumption and collision avoidance, is a non-trivial problem. Traditional approaches

often rely on centralized control or decentralized algorithms, which may not scale well with the increasing number of UAVs. Finally, the interactions between UAVs and their environment, as well as the interactions between UAVs themselves, further complicating the task planning of multi-UAV systems.

This research addresses this critical gap, aiming to revolutionize the way UAV swarms perform tasks by integrating large language models into their control architecture. This paper seeks to contribute to the field of UAV swarm control by proposing a novel approach to task planning using large language models. Specifically, our objectives include investigating the adaptability of LLMs for both synchronous and asynchronous task execution in a swarm, thereby improving overall efficiency and responsiveness. Our approach not only addresses the challenge of coordinating synchronous and asynchronous tasks within a swarm but also establishes a simulation framework for rapid verification, ensuring the practical viability of our proposed method. The successful implementation of LLMs for task planning holds the potential to transform UAV swarms into intelligent and dynamic entities capable of addressing a myriad of real-world challenges, promising a paradigm shift in UAV applications.

To achieve these objectives, the paper is organized as follows. Section II reviews related work in UAV swarm control and language models. Section III details the methodology and system architecture. Section IV presents our experimental results, and Section V concludes the paper.

II. RELATED WORK

Recent breakthroughs in LLMs to understand and generate text has led to a plethora of research works incorporating LLMs into various aspects of robots. We outline LLM for robotics in general and LLM for UAVs in specific for reference.

LLMs for robotics. Since the emergence of various LLMs, including BERT [1], GPT-3 [2], and ChatGPT, their applications in robotics are being widely explored. LLMs have found application in the domain of language-Conditioned imitation learning [3] [4], language-assisted reinforcement learning [5][6], language-image goal-conditioned value learning [7][8], robot task planning [9][10], and LLM-based code generation [11][12]. Further detailed description of foundation models in robotics could be found in some recent survey papers [13], [14], [15], [16].

LLM for UAVs. Researchers in Microsoft outlines a strategy that combines design principles for prompt engi-

*This work was supported by The Major Key Project of Peng Cheng Laboratory under Grants PCL2023A09, and the Talent Program of Guangdong Province (2021QN02Z107).

¹Jinqiang Cui, Guocai Liu, Hui Wang, Yue Yu, Jiankun Yang are with Peng Cheng Laboratory, Shenzhen, China, {cuijq, liugc, wangh06, yuy, yangjk}@pcl.ac.cn

²National University of Defense Technology, Changsha, China

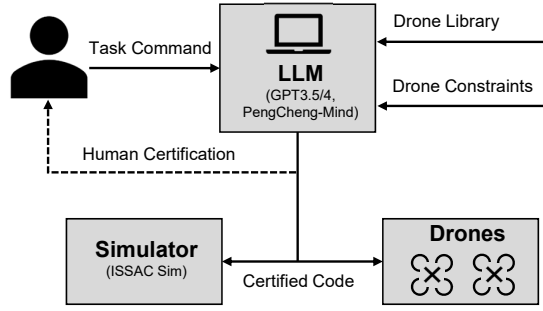


Fig. 1. System diagram of the TPML System

neering and the creation of a high-level function library which allows ChatGPT to adapt to different robotics tasks, simulators, and form factors [11]. They have demonstrated successfully controlled a single UAV in both simulation and real flight tests. Swarm-GPT [17] combines LLMs with a distributed UAV swarm motion planning framework for UAV choreography. AeroAgent [18] presents a novel paradigm for industrial robotic embodied agents, encapsulating an agent as cerebrum, controller as cerebellum architecture.

III. METHOD

A. System Overview

The system diagram of TPML is illustrated in Fig. 1. Human operators input task commands in natural language to the LLM. To transform the LLM into a tailored UAV task manager, it is essential to furnish the LLM with the available UAV code library and relevant constraints, thereby preventing the occurrence of hallucination issues. Leveraging the LLM’s contextual understanding and code generation capabilities, a series of codes are produced. Prior to deploying the code for real flight tests, human certification is mandated to address safety concerns. Once certified, the codes become usable in both the developed simulator and on actual UAV platforms.

B. Human-LLM interaction

Traditional command and control of multi-UAV systems is implemented by ground control stations, where human operators design the flight mission by adding waypoints for each UAV in the system. This can be time-consuming and does not apply to quickly developing scenarios where frequent task re-allocation of the multi-UAV systems are required. Thanks to the recent development of LLMs, instant Human-LLM interaction can be effectively applied to manage various tasks for multi-UAV systems. By leveraging the context understanding and code generation capabilities of LLMs, humans can communicate with UAVs in a more intuitive and natural way, enabling efficient and interactive task descriptions and UAV formation control.

Humans can provide high-level prompts or commands to the LLM to plan and coordinate UAV tasks. During UAV operations, humans can interact with the LLM in real-time to control the UAVs in a changing environment. If the

operator needs to modify or adjust the ongoing UAV tasks, they can provide new prompts to the LLM. The LLM can assist humans in making informed decisions during UAV operations. Humans can rely on LLMs to assess potential safety risks and mitigate them during UAV operations. LLMs can assist in analyzing data collected by UAVs and generating reports. In summary, Human-LLM interaction enhances the control, coordination, and decision-making capabilities in UAV operations. It enables more efficient and intelligent communication between humans and UAVs, leading to improved task execution, safety, and overall operational effectiveness.

C. LLM Model Selection

With the increasing capabilities of various LLM models, the possibility of applying them to robotics are still not fully investigated. For a successful application in robotics, LLMs have to be able to understand human input, make common-sense reasoning, generate corresponding code, and possess zero-shot transfer capability. Even thorough there are a plethora of LLMs available, not all LLMs meet the capability requirements. For state-of-the-art LLM models, including GPT3.5 or GPT4, we could adopt them directly for a fast prototype development. For other LLM models, an extra fine tuning is required.

The fine tuning process includes two stages: the construction of UAV command prompt data for human-machine collaboration and the fine-tuning of the UAV command prompt data of the LLM model. First, we use customized UAV command templates to construct diverse natural language input and the corresponding UAV command output, including single-UAV control instructions, multi-UAV synchronous instructions, and multi-UAV asynchronous instructions. Second, the LLM model is fine-tuned and trained based on the constructed UAV command prompt data so that it can understand the user input and generate the corresponding UAV control code.

D. UAV Motion Library

A UAV motion function library is a collection of pre-defined functions or algorithms that facilitate the control and movement of UAVs. The purpose of a UAV motion function library is to provide a set of reusable and modular components that developers can utilize to build more complex UAV control systems. These libraries often include functions for basic flight maneuvers like takeoff, landing, rotating and altitude control. They may also incorporate more advanced capabilities like path planning, formation flying, or dynamic obstacle avoidance. By using a UAV motion function library, developers can save time and efforts in implementing low-level control functionalities, as they can leverage pre-existing and tested functions. This also constrains the LLM to use the only the functions in the UAV motion library so that the generated code can be recognized and executed by UAVs. Fig. 2 shows the code used in the real flight systems. In order to perform multiple UAV tasks, each function is assigned with an ID to identify each UAV platform.



Here are some functions you can use to command the drone.

fly.takeoff(id) - takes off the id drone. id can be an individual (1,2,...) or 'All'.

fly.land(id) - lands the id drone.

fly.left(x, id) - Fly the id drone x cm left.

fly.right(x, id) - Fly the id drone x cm right.

fly.forward(x, id) - Fly the id drone x cm forward.

fly.back(x, id) - Fly the id drone x cm backwards.

fly.up(x, id) - Fly the id drone x cm upwards.

fly.down(x, id) - Fly the id drone x cm downwards.

fly.flip(x, id) - Flip the id drone to the x direction. x can be 'left', 'right', 'forward', 'back'.

fly.rotate_cw(x, id) - Rotate the id drone x degree clockwise.

fly.rotate_ccw(x, id) - Rotate the id drone x degree counter-clockwise.



Great! Now that we have the list of functions for controlling the Tello drone, let's explore some possible tasks you can accomplish using these functions. Please let me know which specific task you would like to achieve, and I'll provide you with the corresponding Python code and an explanation of what it does.

Fig. 2. The system prompt providing the UAV code library.

E. Prompt Design

Writing well structured prompts is an essential part of ensuring accurate, high quality responses from a language model. In the context of a prompt-driven control system for multi-UAV systems, there are two types of prompts: the system prompts and the user prompts.

The system prompts are predefined instructions or prompts crafted to delineate the functionality of the LLM, as depicted in Fig. 2. These prompts have the capability to alter the role and function of the LLM, transforming it into an expert in a particular domain. They may include constraints and directives such as “You are an assistant aiding me with UAVs,” “Utilize simple Python functions from libraries like math and numpy,” or “You are restricted to using the functions I have defined.” These prompts serve as guiding principles for the LLM, steering its actions to accomplish specific objectives or tasks. A crucial role of the system prompt is to provide the LLM with usable UAV code. This is paramount for UAVs, as only realistic codes recognized and executed by the UAV platforms are employed.

User prompts are prompts created by the user or operator of the multi-UAV system. These prompts are usually in the form of natural language commands or requests provided by the user to the system. These prompts mainly present the executable functions for the UAVs followed by a brief explanation after each function. User prompts can include instructions like “Takeoff the No.1 UAV”, “Inspect a particular structure”, or “Fly the No.1 and No.2 UAVs left simultaneously”. The control system interprets these user prompts and generates appropriate system prompts to guide the UAVs accordingly.

The interaction between system prompts and user prompts forms the basis of the prompt-driven control approach. The system prompts provide the overall guidance and control for the multi-UAV system, while the user prompts allow for flexible and intuitive communication between the user and

the system. The control system analyzes and interprets the user prompts, generating system prompts that translate the user's intent into actionable instructions for the UAVs. By incorporating both system prompts and user prompts, the prompt-driven control system enables effective and efficient control of multi-UAV systems. It combines the advantages of automated control algorithms with the flexibility and adaptability of user input, allowing for dynamic and responsive UAV operations in various scenarios.

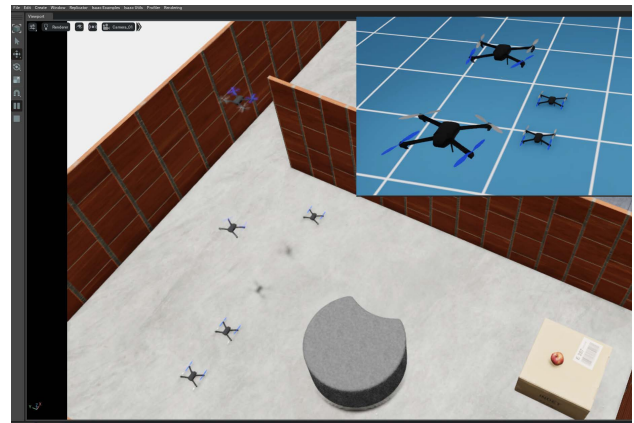


Fig. 3. Simulation Output in ISSAC Sim

F. Simulation Setup

To validate the effectiveness of the TMPL workflow, we have developed a simulation framework for rapid prototyping. This framework, depicted in Fig. 3, consists of four layers, enabling extensive control of unmanned air vehicles by the LLM through direct interaction within the NVIDIA Isaac Sim environment. The initial layer involves the Isaac Sim Environment, capable of importing Universal Scene Description (USD) files through Python scripts. The unmanned air system is represented as a standalone USD file model and

- User: At the same time, all drones takeoff. Drone 1 flies left by 50cm, drone 2 flies right by 50cm. Drone 1 performs a left flip once, and drone 2 performs a right flip once. All drones land.

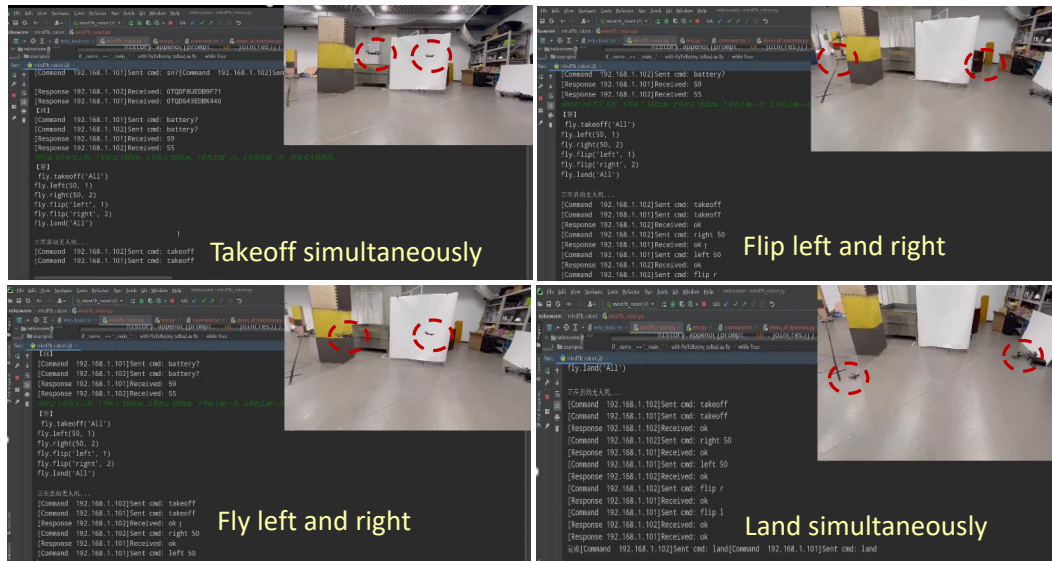


Fig. 4. The multi-UAV synchronous scenario.

- User: Takeoff drone 1, drone 1 flies to the right for 50cm, drone 1 performs a left flip once. After the completion of drone 1's actions, takeoff drone 2. Drone 2 flies to the left for 50cm and performs a right flip once. All drones land.

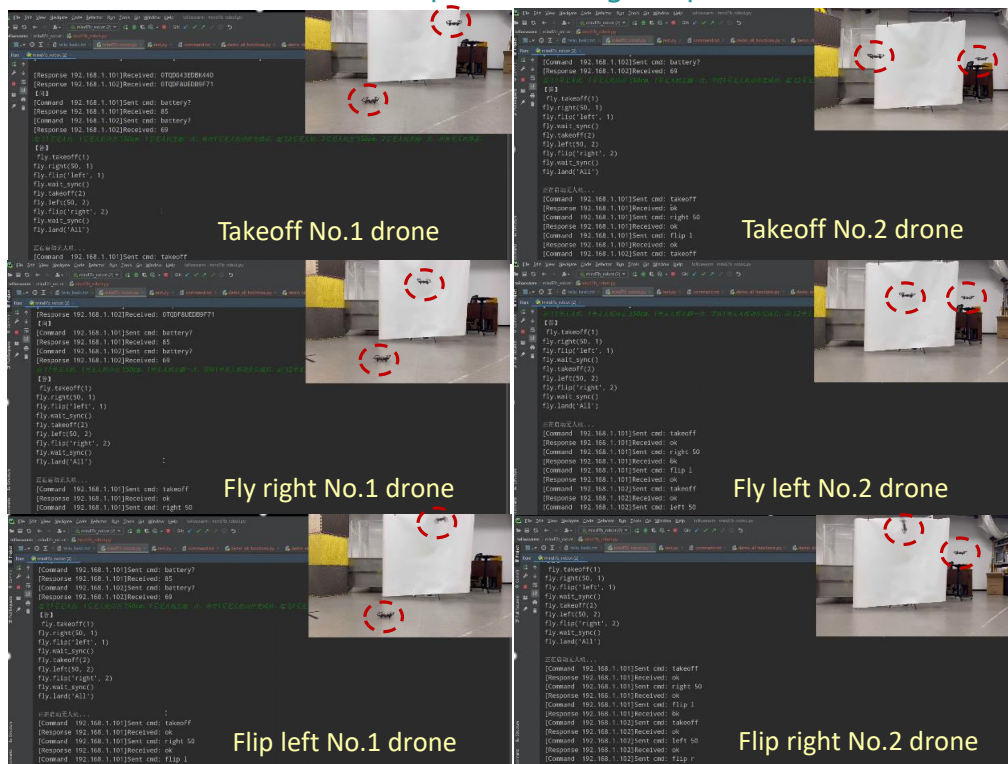


Fig. 5. The multi-UAV asynchronous scenario.

integrated as an Isaac Sim extension, featuring an interactive Graphical User Interface (GUI).

The second layer of the framework incorporates sensor suites and communication functions. It models four types of sensors: barometer, magnetometer, Inertial Measurement Unit (IMU), and Global Positioning System (GPS). Additionally, it includes an Application Programming Interface (API) with callback functions, ensuring smooth access to system states and data generated by each onboard sensor.

The third layer functions as a standalone running process, incorporating communication with the LLMs and the execution of MAVLink or ROS messages. This layer also provides a higher backend API to exert control over UAVs.

The final layer is the LLM layer, responsible for translating user-issued natural language commands into executable function codes. Common LLMs, such as ChatGPT and Pengcheng Mind, are accessible within this layer.

IV. SIMULATION AND EXPERIMENT RESULTS

We assess the efficacy of our proposed approach through a comprehensive evaluation, employing both simulation and real-field flight tests. Initially, we develop a simulation environment to validate the entire operational workflow of TPML. Subsequently, we present the results of flight tests, encompassing both synchronous and asynchronous task sequences involving two UAVs. Fig. 3 shows the simulation output of multiple UAVs in ISSAC Sim environment. A demonstration video is available at Youtube¹. The simulation framework is open source at Github².

For real flight verification, we employ small and lightweight Tello UAVs known for their programming APIs, facilitating fast and straightforward deployment. Two Tello UAVs are configured in Access Point mode and connected to the same wireless network as the laptop hosting the LLM interface. Each UAV is assigned a unique ID through an IP address, enabling the LLM interface to connect to each UAV individually. Utilizing the LLM's contextual understanding and code generation capabilities, the interface processes task commands in text or voice format, converting them into executable UAV code. The generated codes are automatically transmitted to the UAVs for real flight execution. To assess the effectiveness of TPML, we conducted separate experiments evaluating both synchronous and asynchronous control of the two UAVs.

In the synchronous scenario, depicted in Fig.4, the user issues task inputs using the phrase "At the same time...", directing both UAVs to execute tasks concurrently. The generated codes coordinate the simultaneous execution of specified actions for both UAVs. These actions encompass the takeoff of both UAV 1 and UAV 2 (top-left figure), lateral movement to the left and right (bottom-left figure), flipping maneuvers to the left and right (top-right figure), and simultaneous landing (bottom-right figure).

In the asynchronous scenario, illustrated in Fig.5, the UAV action sequence follows a temporal order indicated

by phrases like "After the completion of UAV 1." User tasks in this context exhibit a clear temporal sequence, with actions for UAV 2 dependent on the completion of UAV 1's actions. TPML demonstrates its capability to manage tasks with distinct time constraints in this experiment.

Synchronous Scenario	Asynchronous Scenario
【答】 <pre>fly.takeoff('All') fly.left(50, 1) fly.right(50, 2) fly.flip('left', 1) fly.flip('right', 2) fly.land('All')</pre>	【答】 <pre>fly.takeoff(1) fly.right(50, 1) fly.flip('left', 1) fly.wait_sync() fly.takeoff(2) fly.left(50, 2) fly.flip('right', 2) fly.wait_sync() fly.land('All')</pre>

Fig. 6. Code generated by TPML for synchronous and asynchronous tasks

For both synchronous and asynchronous tasks, the code snippets generated by TPML are explicitly displayed in Fig.6. In the synchronous task, the codes for both UAVs are executed sequentially. For the asynchronous task, a special function *fly.wait_sync* occur at two places. The first one indicates that UAV 2 can only takeoff (*fly.takeoff(2)*) after UAV 1 finish the left flip function (*fly.flip('left',1)*). The second one indicates the two UAVs should only execute land function (*fly.land('All')*) until UAV 2 finish the right flip action (*fly.flip('right',2)*).

V. CONCLUSION

In summary, we introduce TPML, a pipeline designed to understand human language input and extract executable codes from LLMs, facilitating the planning of both synchronous and asynchronous tasks for a swarm of UAVs. TPML is versatile, accommodating both commercial off-the-shelf and fine-tuned LLMs, and we offer an open-source simulation framework for the benefit of the community. The effectiveness of TPML is validated through a combination of experiments and simulation results, suggesting a promising avenue for multi-UAV systems task planning. Our future endeavors involve integrating vision inference results from multiple UAVs and leveraging LLMs to extract semantic information for situation assessment.

NOMENCLATURE

<i>API</i>	Application Programming Interface
<i>GPS</i>	Global Positioning System
<i>GUI</i>	Graphical User Interface
<i>IMU</i>	Inertial Measurement Unit
<i>LLM</i>	Large Language Model
<i>UAV</i>	Unmanned Aerial Vehicle
<i>USD</i>	Universal Scene Description

¹https://youtu.be/YcNkwo_PNNq

²<https://github.com/PengICS/cai-sim>

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North*, Jan 2019. [Online]. Available: <http://dx.doi.org/10.18653/v1/n19-1423>
- [2] T. Brown, B. Mann, and e. a. Ryder, Nick, "Language models are few-shot learners," *arXiv: Computation and Language*, *arXiv: Computation and Language*, May 2020.
- [3] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet, "Learning latent plans from play," *Conference on Robot Learning (CoRL)*, 2019. [Online]. Available: <https://arxiv.org/abs/1903.01973>
- [4] C. Lynch and P. Sermanet, "Language conditioned imitation learning over unstructured data," *Robotics: Science and Systems*, 2021. [Online]. Available: <https://arxiv.org/abs/2005.07648>
- [5] N. Di Palo, A. Byravan, L. Hasenclever, M. Wulfmeier, N. Heess, and M. Riedmiller, "Towards A Unified Agent with Foundation Models," *arXiv e-prints*, p. arXiv:2307.09668, July 2023.
- [6] Y. Du, O. Watkins, Z. Wang, C. Colas, T. Darrell, P. Abbeel, A. Gupta, and J. Andreas, "Guiding Pretraining in Reinforcement Learning with Large Language Models," *arXiv e-prints*, p. arXiv:2302.06692, Feb. 2023.
- [7] M. Ahn, A. Brohan, Y. Chebotar, and et al, "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances," in *6th Annual Conference on Robot Learning*, 2022.
- [8] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models," in *7th Annual Conference on Robot Learning*, 2023.
- [9] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in *International Conference on Machine Learning*. PMLR, 2022, pp. 9118–9147.
- [10] Y. Ding, X. Zhang, C. Paxton, and S. Zhang, "Task and motion planning with large language models for object rearrangement," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 2086–2092.
- [11] S. Vemprala, R. Bonatti, A. Buckner, and A. Kapoor, "Chatgpt for robotics: Design principles and model abilities," Tech. Rep. MSR-TR-2023-8, February 2023. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/chatgpt-for-robotics-design-principles-and-model-abilities/>
- [12] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 9493–9500.
- [13] R. Firoozi, J. Tucker, S. Tian, A. Majumdar, J. Sun, W. Liu, Y. Zhu, S. Song, A. Kapoor, K. Hausman, B. Ichter, D. Driess, J. Wu, C. Lu, and M. Schwager, "Foundation Models in Robotics: Applications, Challenges, and the Future," *arXiv e-prints*, p. arXiv:2312.07843, Dec. 2023.
- [14] J. Wang, Z. Wu, and e. a. Li, Yiwei, "Large Language Models for Robotics: Opportunities, Challenges, and Perspectives," *arXiv e-prints*, p. arXiv:2401.04334, Jan. 2024.
- [15] R. Bommasani, D. A. Hudson, E. Adeli, and e. a. Altman, "On the Opportunities and Risks of Foundation Models," *arXiv e-prints*, p. arXiv:2108.07258, Aug. 2021.
- [16] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," *arXiv e-prints*, p. arXiv:2402.01680, Jan. 2024.
- [17] A. Jiao, T. P. Patel, S. Khurana, A.-M. Korol, L. Brunke, V. K. Adajania, U. Culha, S. Zhou, and A. P. Schoellig, "Swarm-GPT: Combining Large Language Models with Safe Motion Planning for Robot Choreography Design," *arXiv e-prints*, p. arXiv:2312.01059, Dec. 2023.
- [18] H. Zhao, F. Pan, H. Ping, and Y. Zhou, "Agent as Cerebrum, Controller as Cerebellum: Implementing an Embodied LMM-based Agent on Drones," *arXiv e-prints*, p. arXiv:2311.15033, Nov. 2023.